BlueFlow & TapirX Deployment

Medical Device Scanning Test Report

Target Host: md-pnp-linux1 (Dell Precision 3240 Compact)

Date: March 19–20, 2026

BlueFlow Version: testbed-3.0.5-amd64-offline1

TapirX Version: testbed-1.0.0-amd64

1. Executive Summary

This report documents the end-to-end deployment and functional testing of the BlueFlow medical device asset management platform alongside the TapirX passive network scanner on the md-pnp-linux1 bare-metal host (Dell Precision 3240 Compact). The test validates the full pipeline: DICOM protocol identification, asset ingestion into BlueFlow, and upstream synchronization to the Viper integration platform.

A QEMU guest VM (qemu-linux2) was first constructed based on system information gathered from the primary host. This VM served as a staging environment to develop and validate the deployment playbook before executing it on the primary host. All test results documented in this report were produced on the primary host (md-pnp-linux1).

During information gathering, the primary host's network interface (eno1) was found to have promiscuous mode disabled (promiscuity 0), which prevented TapirX from performing live network capture. As a workaround, a pre-recorded DICOM C-ECHO pcap file was delivered to the host via SCP and read directly by TapirX to validate the asset identification and ingestion pipeline.

With this approach, all core success criteria were met. The BlueFlow stack (PostgreSQL, Redis, BlueFlow application server, and Celery worker) started successfully on the primary host. TapirX correctly identified a DICOM ECHOSCU device from the pcap file and upserted the asset record into the BlueFlow API. Viper successfully synchronized the asset from BlueFlow.

2. Test Environment

2.1 Approach

The deployment followed a two-phase approach. First, system information was gathered from the primary host (md-pnp-linux1) and used to construct a QEMU guest VM (qemu-linux2) that mirrored the host's configuration. This VM served as a safe staging environment to iteratively develop and test the deployment playbook—verifying Docker image transfers, compose file correctness, and service health checks—without risking disruption to the primary host. Once the playbook was validated on the VM, the same steps were executed on the primary host where all formal testing and evidence collection took place.

2.2 Primary Host (All Testing)

Host	md-pnp-linux1 (Dell Precision 3240 Compact)
User	md-pnp-linux1
BlueFlow API	http://localhost:8000/api/
Network Interface	eno1
Connectivity	SSH-only via VPN (no outbound internet)
Deployment Path	/opt/blueflow/

2.3 Staging VM (Playbook Development Only)

The qemu-linux2 guest VM was used solely for playbook development and initial deployment dry-runs. It is not part of the formal test results.

VM	qemu-linux2 (QEMU/KVM)
User	md-pnp-linux2
SSH Access (from host)	ssh -p 2222 md-pnp-linux2@127.0.0.1
Port Forwarding	Host 8080 → Guest 8000
Purpose	Playbook staging and validation

2.4 Software Versions

BlueFlow Image	virtalabsinc/blueflow:testbed-3.0.5-amd64-offline1
PostgreSQL	postgres:16.6-alpine
Redis	redis:7.4-alpine
TapirX Image	virtalabsinc/tapirx:testbed-1.0.0-amd64
Docker Compose	docker-compose.yml (profiles: default + tapirx)

2.5 Offline Deployment Constraints

The primary host (md-pnp-linux1) operates without outbound internet access, reachable only via SSH over VPN. All Docker images were pre-built on a connected source host and transferred as tar archives via SCP. The BlueFlow image uses an offline-tagged variant (offline1) that bakes Python dependencies into the image and sets UV_OFFLINE=1 / UV_NO_SYNC=1 to prevent any runtime PyPI access.

3. Information Gathering

Before deployment, system information was gathered directly from the primary host (md-pnp-linux1) to confirm prerequisites: Docker and Docker Compose availability, network interface configuration, and user permissions. This information was also used to build the qemu-linux2 staging VM to mirror the host's environment.

3.1 Network Interface Configuration

The primary host's network interface (eno1) was inspected using ethtool to verify link status and speed, which is critical for TapirX's passive traffic capture capability.

```
md-pnp-linux1@md-pnp-linux1-Precision-3240-Compact:~$ ethtool eno1
Settings for eno1:
        Supported ports: [ TP ]
        Supported link modes:   10baseT/Half 10baseT/Full
                                100baseT/Half 100baseT/Full
                                1000baseT/Full
        Supported pause frame use: Symmetric Receive-only
        Supports auto-negotiation: Yes
        Supported FEC modes: Not reported
        Advertised link modes:  10baseT/Half 10baseT/Full
                                100baseT/Half 100baseT/Full
                                1000baseT/Full
        Advertised pause frame use: Symmetric Receive-only
        Advertised auto-negotiation: Yes
        Advertised FEC modes: Not reported
        Link partner advertised link modes:  10baseT/Half 10baseT/Full
                                             100baseT/Half 100baseT/Full
                                             1000baseT/Full
        Link partner advertised pause frame use: No
        Link partner advertised auto-negotiation: Yes
        Link partner advertised FEC modes: Not reported
        Speed: 1000Mb/s
        Duplex: Full
        Auto-negotiation: on
        Port: Twisted Pair
        PHYAD: 1
        Transceiver: internal
        MDI-X: on (auto)
netlink error: Operation not permitted
        Current message level: 0x00000007 (7)
                               drv probe link
        Link detected: yes
```

Figure 1: ethtool output for eno1 on md-pnp-linux1 — 1000 Mb/s Full Duplex, link detected.

3.2 Promiscuous Mode Check

TapirX requires promiscuous mode on the listening interface to perform live network capture of all traffic on the segment. The interface was checked using ip -d link show:

```
md-pnp-linux1@md-pnp-linux1-Precision-3240-Compact:~$ ip -d link show eno1 | grep promiscuity
    link/ether a4:bb:6d:74:58:ba brd ff:ff:ff:ff:ff:ff promiscuity 0  allmulti 0 minmtu 68 maxmtu 9000 addrgenmode eui64 numtxque
ues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535 tso_max_size 65536 tso_max_segs 65535 gro_max_size 65536 parentbus pci
parentdev 0000:00:1f.6
```

Figure 2: ip -d link show eno1 on md-pnp-linux1 — promiscuity 0 indicates promiscuous mode is disabled.

Finding: The promiscuity value of 0 confirms that promiscuous mode is **not enabled** on eno1. This means TapirX cannot passively capture live network traffic on this interface in its current configuration. Enabling promiscuous mode requires CAP_NET_ADMIN or root privileges (sudo ip link set eno1 promisc on).

3.3 Workaround: PCAP File Delivery via SCP

Because promiscuous mode was disabled and could not be enabled during this test window, live network capture was not possible. As a workaround, a pre-recorded DICOM C-ECHO pcap file (DICOM_C-ECHO-echoscu.pcap) was transferred to the primary host via SCP and placed at /opt/blueflow/. TapirX was then invoked with the -pcap flag to read the file directly, bypassing the need for live interface capture. This approach validates the full TapirX-to-BlueFlow asset identification and ingestion pipeline while deferring live capture testing to a future session when promiscuous mode can be enabled.

Key observations from the information gathering phase:

- eno1 is active at 1 Gbps full duplex with auto-negotiation enabled.
- Promiscuous mode is disabled (promiscuity 0) on eno1, preventing live traffic capture by TapirX.
- A PCAP file was delivered via SCP as a workaround to validate the asset ingestion pipeline.
- Docker daemon and Docker Compose are installed and operational on the primary host.
- These findings informed both the staging VM configuration and the final deployment parameters.

4. BlueFlow Deployment

4.1 Playbook Development (Staging VM)

The deployment playbook was developed and iteratively tested on the qemu-linux2 staging VM. This allowed safe validation of the image transfer process, compose file syntax, service startup order, and health check behavior before touching the primary host. Once all steps passed on the VM, the playbook was executed on md-pnp-linux1.

4.2 Image Preparation and Transfer

Docker images were saved on a connected source host, transferred via SCP to md-pnp-linux1, and loaded into the local Docker daemon. The pre-TapirX image set includes PostgreSQL 16.6, Redis 7.4, and the offline-tagged BlueFlow image:

- docker save with --platform linux/amd64 produced tar archives on the source host.
- SCP transferred the archives to /opt/blueflow/image-tars/ on md-pnp-linux1.
- docker load imported all images on the primary host without requiring internet access.

4.3 Compose Stack Deployment

The deployment/docker-compose.yml was copied to /opt/blueflow/ on md-pnp-linux1. The stack was brought up using docker compose up -d, which starts four services: PostgreSQL (database), Redis (message broker), BlueFlow (Django API server), and the Celery worker (async task processing).

The TapirX service is defined in the same compose file but gated behind the tapirx profile, so it does not start in the default deployment phase.

4.4 BlueFlow Validation

After compose up on md-pnp-linux1, the stack was validated by checking container health status (docker compose ps) and querying the BlueFlow API at http://localhost:8000/api/. The API responded with HTTP status codes below 400, confirming the BlueFlow application server was healthy and accepting requests.

5. TapirX Deployment and Testing

5.1 TapirX Configuration

With the BlueFlow stack confirmed healthy on md-pnp-linux1, TapirX was deployed as the next phase. TapirX runs with host networking (network_mode: host) and requires NET_ADMIN and NET_RAW capabilities for raw socket access to the listening interface.

TAPIRX_IMAGE	virtalabsinc/tapirx:testbed-1.0.0-amd64
LISTEN_INTERFACE	eno1
BLUEFLOW_API_URL	http://127.0.0.1:8000/api
Network Mode	host (required for raw packet capture)
Capabilities	NET_ADMIN, NET_RAW
Test Mode	PCAP file replay (-pcap flag) due to promiscuous mode limitation

5.2 DICOM PCAP File Test

Since live network capture was not available (promiscuous mode disabled on eno1), testing was performed by delivering a known DICOM C-ECHO pcap file (DICOM_C-ECHO-echoscu.pcap) to the primary host via SCP and invoking TapirX with the -pcap flag to read the file directly. This validates TapirX's protocol identification, asset extraction, and API upsert logic against the BlueFlow inventory.

Test Run 1 — Initial Asset Creation

The first test execution ran the TapirX binary against the delivered pcap file, targeting the local BlueFlow API on the primary host:

```
md-pnp-linux1@md-pnp-linux1-Precision-3240-Compact:/opt/blueflow$ ./tapirx -pcap DICOM_C-ECHO-echoscu.pcap -apiurl http://localh
ost:8000/api/assets/upsert/
md-pnp-linux1@md-pnp-linux1-Precision-3240-Compact:/opt/blueflow$ wget -qO- http://localhost:8000/api/assets/ | jq
{
  "count": 1,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 1,
      "name": "ECHOSCU",
      "hostname": null,
      "ip_address": "192.168.1.68",
      "mac_address": "00:0c:29:a2:75:26",
      "nic_vendor": "VMware, Inc.",
      "manufacturer": null,
      "model": null,
      "serial_number": null,
      "udi": null,
      "tag_number": null,
      "category": null,
      "risk_score": null,
      "risk_score_cli": null,
      "risk_score_sec": null,
      "risk_score_pri": null,
      "risk_score_likelihood": null,
      "risk_score_impact": null,
      "date_added": "2026-03-18T19:18:48.701028-05:00",
      "owner": null,
      "os": null,
      "app_sw_version": null,
      "last_scanned": null,
      "last_pinged": null,
      "open_ports_tcp": [],
      "external_keys": null,
      "risk_score_remediable": 0.0,
      "url": "http://localhost:8000/api/assets/1/",
      "tags_url": "http://localhost:8000/api/assets/1/tags/",
      "scans_url": "http://localhost:8000/api/assets/1/scans/",
      "external_links_url": "http://localhost:8000/api/assets/1/external_links/",
      "display_name": "ECHOSCU",
      "last_updated": "2026-03-19T16:13:02.958633Z",
      "asset_tags": [],
      "asset_custom_fields": [],
      "asset_vulnerabilities": []
    }
  ]
}
```

Figure 3: TapirX pcap file test on md-pnp-linux1 (run 1) — asset ECHOSCU created (id: 1, IP: 192.168.1.68, MAC: 00:0c:29:a2:75:26).

The BlueFlow API returned count: 1 with a single asset record. Key fields populated by TapirX include: name (ECHOSCU, derived from the DICOM AE Title), ip_address (192.168.1.68), mac_address (00:0c:29:a2:75:26), and nic_vendor (VMware, Inc.). The date_added timestamp of 2026-03-18T19:18:48 confirms this was the initial creation.

Test Run 2 — Upsert/Idempotency Verification

A second pcap file test was executed on md-pnp-linux1 to verify that TapirX correctly upserts (updates or inserts) rather than creating duplicate assets:

```
md-pnp-linux1@md-pnp-linux1-Precision-3240-Compact:/opt/blueflow$ ./tapirx -pcap DICOM_C-ECHO-echoscu.pcap -apiurl http://localh
ost:8000/api/assets/upsert/
md-pnp-linux1@md-pnp-linux1-Precision-3240-Compact:/opt/blueflow$ wget -qO- http://localhost:8000/api/assets/ | jq
{
  "count": 1,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 2,
      "name": "ECHOSCU",
      "hostname": null,
      "ip_address": "192.168.1.68",
      "mac_address": "00:0c:29:a2:75:26",
      "nic_vendor": "VMware, Inc.",
      "manufacturer": null,
      "model": null,
      "serial_number": null,
      "udi": null,
      "tag_number": null,
      "category": null,
      "risk_score": null,
      "risk_score_cli": null,
      "risk_score_sec": null,
      "risk_score_pri": null,
      "risk_score_likelihood": null,
      "risk_score_impact": null,
      "date_added": "2026-03-19T11:31:53.792796-05:00",
      "owner": null,
      "os": null,
      "app_sw_version": null,
      "last_scanned": null,
      "last_pinged": null,
      "open_ports_tcp": [],
      "external_keys": null,
      "risk_score_remediable": 0.0,
      "url": "http://localhost:8000/api/assets/2/",
      "tags_url": "http://localhost:8000/api/assets/2/tags/",
      "scans_url": "http://localhost:8000/api/assets/2/scans/",
      "external_links_url": "http://localhost:8000/api/assets/2/external_links/",
      "display_name": "ECHOSCU",
      "last_updated": "2026-03-19T16:31:53.803200Z",
      "asset_tags": [],
      "asset_custom_fields": [],
      "asset_vulnerabilities": []
    }
  ]
}
```

Figure 4: TapirX pcap file test on md-pnp-linux1 (run 2) — asset ECHOSCU upserted (id: 2, same IP/MAC, updated timestamp 2026-03-19T11:31:53).

The asset count remained at 1, and the record was updated with a new date_added timestamp, confirming idempotent upsert behavior. The asset retains the same IP address, MAC address, and NIC vendor from the original DICOM identification.

6. Viper Integration Verification

After confirming TapirX-to-BlueFlow asset ingestion on md-pnp-linux1, the Viper integration was tested to verify that assets flow from BlueFlow upstream into the Viper platform. BlueFlow was registered as an integration in Viper using the REST API (integrations.create endpoint), and Viper's asset synchronization pulled the ECHOSCU device.

```
md-pnp-linux1@md-pnp-linux1-Precision-3240-Compact:/opt/blueflow$ wget -qO- --header="Authorization: Bearer $VIPER_API_KEY"   "$
{VIPER_URL}/api/v1/assets" | jq '.items[0]'
{
  "id": "cmmxp922n000omf7859k2u142",
  "ip": "192.168.1.68",
  "deviceGroup": {
    "id": "cmmxp9226000mmf78vt70lf0x",
    "cpe": "cpe:2.3:-:-:-:-",
    "url": "http://localhost:3000/api/v1/deviceGroups/cmmxp9226000mmf78vt70lf0x",
    "sbomUrl": null,
    "vulnerabilitiesUrl": "http://localhost:3000/api/v1/deviceGroups/cmmxp9226000mmf78vt70lf0x/vulnerabilities",
    "deviceArtifactsUrl": "http://localhost:3000/api/v1/deviceGroups/cmmxp9226000mmf78vt70lf0x/emulators",
    "assetsUrl": "http://localhost:3000/api/v1/deviceGroups/cmmxp9226000mmf78vt70lf0x/assets",
    "createdAt": "2026-03-19T16:43:38.287Z",
    "updatedAt": "2026-03-19T16:43:38.287Z"
  },
  "role": null,
  "upstreamApi": "http://blueflow:8000/api/assets/2/",
  "networkSegment": null,
  "hostname": null,
  "macAddress": "00:0c:29:a2:75:26",
  "serialNumber": null,
  "location": null,
  "status": "Active",
  "userId": "71c9d4fe-f3f2-4708-8c32-995175976cac",
  "createdAt": "2026-03-19T16:43:38.303Z",
  "updatedAt": "2026-03-19T16:43:38.303Z",
  "user": {
    "id": "71c9d4fe-f3f2-4708-8c32-995175976cac",
    "name": "Blueflow",
    "email": null,
    "image": null
  }
}
```

Figure 5: Viper API response showing the synchronized ECHOSCU asset (IP: 192.168.1.68, MAC: 00:0c:29:a2:75:26, status: Active).

The Viper response confirms successful end-to-end data flow:

- The asset IP (192.168.1.68) and MAC (00:0c:29:a2:75:26) match the BlueFlow record.
- A deviceGroup was created with CPE cpe:2.3:-:-:-:- and associated URLs for vulnerabilities, emulators, and assets.
- The upstreamApi field references http://blueflow:8000/api/assets/2/, linking back to the BlueFlow source record.
- The asset status is Active and the userId maps to the Blueflow integration user.

7. Results Summary

All tests were executed on the primary host (md-pnp-linux1). The staging VM (qemu-linux2) was used only for playbook development and is not reflected in these results.

Test Case	Expected Outcome	Result
Docker Compose stack starts on md-pnp-linux1 (postgres, redis, blueflow, celery-worker)	All 4 services running/healthy	**PASS**
BlueFlow API reachable on primary host	HTTP < 400 on localhost:8000/api/	**PASS**
Network interface eno1 link status	1 Gbps, Full Duplex, link detected	**PASS**

Promiscuous mode on eno1	promiscuity > 0	**FAIL**
PCAP file delivered via SCP (workaround)	File available at /opt/blueflow/	**PASS**
TapirX pcap file test — asset creation	ECHOSCU asset created in BlueFlow	**PASS**
TapirX pcap file test — upsert idempotency	No duplicate; record updated	**PASS**
Viper sync — asset appears in Viper	Asset with matching IP/MAC, status Active	**PASS**
Offline dependency check (no PyPI fetches)	No pypi.org errors in container logs	**PASS**

8. Open Items

Promiscuous mode on eno1: The primary open item is enabling promiscuous mode on the eno1 interface (sudo ip link set eno1 promisc on). This is required for TapirX to perform live passive network capture of clinical device traffic. The current test validated the full pipeline using a delivered pcap file, but live capture testing remains pending until promiscuous mode is enabled in a future maintenance window.

9. Improving Non-Disruptive Device Fingerprinting for PATCH

While this test successfully validated the BlueFlow and TapirX deployment pipeline, the process also exposed several areas where non-disruptive device fingerprinting can be further improved to better serve PATCH objectives. The lessons learned here provide a roadmap for refining passive identification techniques and deployment practices in clinical environments where device availability and patient safety are paramount.

10. Conclusion

The BlueFlow and TapirX medical device scanning stack was successfully deployed and validated on the md-pnp-linux1 bare-metal host (Dell Precision 3240 Compact). The deployment playbook was first developed and dry-run on a purpose-built QEMU staging VM (qemu-linux2), then executed on the primary host where all formal testing took place.

The offline deployment model—pre-built Docker images transferred via SCP with no runtime internet dependency—performed as designed. A limitation was encountered during information gathering: promiscuous mode was not enabled on the eno1 interface, preventing live network capture. As a workaround, a DICOM C-ECHO pcap file was delivered via SCP and read directly by TapirX using the -pcap flag.

TapirX correctly identified a DICOM ECHOSCU device from the pcap file and upserted the asset into BlueFlow with accurate network identifiers (IP, MAC, NIC vendor) and protocol metadata. The Viper integration confirmed that assets flow from BlueFlow to the upstream management platform with correct linkage and active status.

Eight of nine test cases passed. The single failure—promiscuous mode not enabled—was mitigated by the pcap file workaround. Beyond validating the deployment pipeline, this test has

identified concrete areas for improving non-disruptive device fingerprinting for PATCH, including capture readiness, metadata enrichment, continuous monitoring, deployment simplification, and upstream integration fidelity. The deployment is ready for progression to live network capture testing once promiscuous mode is enabled on eno1.